

ADR-001 · Keep one comparable script and result directory per loader

Field	Detail
Decision	Maintain separate Docling, Unstructured, and Marker scripts that process the same chapter PDF and write to loader-specific directories.
Date	2026-07-14
Context	The project needed to explore multiple document loaders while preserving outputs for side-by-side inspection.
Options Considered	One configurable abstraction over all loaders (less duplication, but hides library-specific behavior) · Separate scripts with shared inputs (simple and transparent) · One-off CLI commands (fastest initially, but not repeatable)
Chosen Solution	Use <code>docling_test.py</code> , <code>unstructured_test.py</code> , and <code>marker_test.py</code> , with outputs under <code>docling_results/</code> , <code>unstructured_results/</code> , and <code>marker_results/</code> .
Rationale	Small independent scripts expose each library's native API and failure modes while keeping the corpus constant for comparison.
Impact	Establishes the current high-level architecture, creates some repeated source/path code, and enables Research topic 1.

ADR-002 · Standardize the Windows environment on Python 3.12

Field	Detail
Decision	Use Python 3.12 instead of Python 3.14 for every loader environment.
Date	2026-07-14
Context	Marker required <code>regex==2024.11.6</code> , which had no compatible prebuilt Windows wheel for Python 3.14 and attempted a source build without MSVC Build Tools. Similar native dependencies made further 3.14 issues likely.
Options	Stay on Python 3.14 and install build tools (large setup and continued compatibility risk) · Maintain separate runtimes per

Considered	loader (works but fragments the project) · Move all loaders to Python 3.12 (mature wheel support and one environment)
Chosen Solution	Use Python 3.12.10 with <code>uv</code> and retire the Python 3.14 environment. ADR-005 later defines separate environments because the selected packages cannot coexist in one resolver solution.
Rationale	Python 3.12 provided compatible wheels for the native ML/OCR stack. Environment isolation is a separate dependency-compatibility decision recorded in ADR-005.
Impact	Resolved BUG-001 and BUG-002, defines the runtime in <code>Architecture.md</code> , and affects all three scripts.

ADR-003 · Use PowerShell as the verified Windows execution shell

Field	Detail
Decision	Execute and validate the Windows virtual-environment scripts through PowerShell.
Date	2026-07-14
Context	Git Bash reported exit 139/127 and a segmentation fault around the Unstructured native dependency stack, while the same interpreter import and full conversion succeeded in PowerShell.
Options Considered	Debug Git Bash/MSYS process behavior · Use PowerShell, the native project shell · Run under WSL with a separate Linux environment
Chosen Solution	Use commands such as <code>& ".venv\Scripts\python.exe" unstructured_test.py</code> from PowerShell for Windows verification.
Rationale	PowerShell exercised the actual Windows interpreter and DLL environment reliably and produced successful output files.
Impact	Closes BUG-004 operationally and is documented as a runtime constraint in <code>Architecture.md</code> .

ADR-004 · Batch every source file and mirror relative output paths

Field	Detail
Decision	Each loader recursively processes every file under <code>source/</code> and mirrors the source-relative path as Markdown under its own result directory.
Date	2026-07-15
Context	The three scripts had diverged into hard-coded filenames and working-directory-dependent paths, preventing repeatable batch comparisons.
Options Considered	Keep one hard-coded file per script (simple but manual) · Accept command-line paths (flexible but requires orchestration) · Use a shared recursive source directory (repeatable and zero-argument)
Chosen Solution	Resolve the project root from <code>file</code> , discover <code>source/**/*</code> , reuse one converter per batch where applicable, create mirrored output directories, continue after per-file errors, and return a nonzero status for partial failure.
Rationale	A shared zero-argument corpus keeps all three loader runs comparable while supporting nested source organization and execution from any working directory.
Impact	Changes all scripts under <code>codes/</code> , supersedes the directory-creation limitation in BUG-003, and makes unsupported-format failures visible without discarding successful conversions.

ADR-005 · Isolate each loader in its own Python environment

Field	Detail
Decision	Run Docling, Unstructured, and Marker from <code>.venv-docling</code> , <code>.venv-unstructured</code> , and <code>.venv-marker</code> respectively.
Date	2026-07-15
Context	A combined installation was unsatisfiable: Unstructured 0.24.1's PDF stack requires <code>pi-heif</code> , whose available releases require Pillow 11.1 or newer, while Marker PDF 1.10.2 requires Pillow below 11. The shared environment also accumulated incomplete Torch metadata and an unrelated <code>marker</code> distribution.
Options Considered	Force one shared environment (resolver conflict) · Change loader versions until constraints overlap (would change the comparison baseline) · Use one Python 3.12 environment per loader (more setup, deterministic isolation)
Chosen Solution	Create three clean Python 3.12 environments with direct <code>uv pip install</code> commands and invoke each script with its matching interpreter. Install <code>marker-pdf</code> , not the unrelated <code>marker</code> package.
Rationale	Isolation preserves the selected comparison versions and prevents incompatible or damaged native dependency stacks from contaminating the other loaders.
Impact	Supersedes the shared-environment portion of ADR-002, removes the combined <code>requirements.txt/uv.lock</code> workflow, and resolves BUG-005 through BUG-008 operationally.

ADR-006 · Scope chunking to Marker Markdown and keep run reports untracked

Field	Detail
Decision	The chunking workflow reads Markdown recursively from <code>marker_results/</code> only, preserves source identity and per-source sequence numbers, and writes timestamped diagnostic reports under the Git-ignored <code>chunk-runs/</code> directory.
Date	2026-07-16
Context	The project moved from loader comparison toward testing downstream ingestion. It needed repeatable chunks from the evaluated Marker output without mixing other loaders or committing multi-megabyte derived run logs.
Options Considered	Chunk every loader result together (broader but confounds parser comparisons) · Accept arbitrary input paths (flexible but less repeatable) · Limit the experiment to Marker Markdown and isolate run artifacts (narrow and auditable)
Chosen Solution	Add <code>chunking/ingest.py</code> and <code>chunking/logger_config.py</code> ; discover <code>.md</code> and <code>.markdown</code> below <code>marker_results/</code> , attach source and sequence metadata, and emit one formatted report per execution below <code>chunk-runs/</code> .
Rationale	A fixed input family makes chunking runs comparable, while source metadata preserves traceability and ignored reports allow frequent experimentation without inflating repository history.
Impact	Establishes the ingestion boundary and report location. It does not adopt a final splitting algorithm: the committed recursive splitter is a baseline, and the later custom heading/table/list prototype was rejected for replacement.

ADR-007 · Normalize Marker clause structure in memory before chunking

Field	Detail
Decision	Add a <code>preprocessing()</code> pass in <code>chunking/ingest.py</code> that repairs Marker's inconsistent numbered-clause and heading formatting per document, in memory, immediately before <code>temp_split()</code> , without rewriting the Marker Markdown on disk.
Date	2026-07-20
Context	Marker renders one logical numbered clause (for example 5.1) inconsistently as a plain paragraph, a dash-prefixed list item, or a bold-labelled line, and wraps a single clause's body across stray blank lines from source pagination. <code>temp_split()</code> chooses chunk boundaries from heading and list structure, so this inconsistency degraded chunking. The font size/weight signal that marked these as headings in the PDF is already gone by the time Marker emits Markdown, and bold is overloaded for inline defined terms, so headings cannot be recovered by a naive "promote bold" rule. The project principle (CLAUDE.md) is that "chunking is not a repair mechanism for source-conversion defects."
Options Considered	Boundary-only soft-heading detection inside the splitter, feeding extra whole-line-anchored patterns to <code>heading_matches()</code> without altering text (honors the principle strictly, but cannot rejoin fragmented clause bodies or supply missing list structure) · In-memory normalization before splitting that rewrites clause structure but never touches the <code>.md</code> files (repairs structure for the splitter while keeping the on-disk output authoritative) · Rewriting <code>marker_results/*.md</code> on disk (simplest downstream, but silently mutates the Marker output still under evaluation and destroys the audit trail)
Chosen Solution	Apply <code>preprocessing(document.page_content)</code> inside <code>split_documents()</code> before <code>temp_split()</code> . It runs <code>normalize_marker_sequences()</code> , <code>promote_italic_sublabels()</code> , and <code>extract_leading_spans()</code> on the in-memory text only; the on-disk Marker Markdown is left unchanged. Scope is limited to the numbered-clause, bold-label, italic-sublabel, and leading <code></code> anchor patterns observed in the RFT/contract document family.
Rationale	In-memory normalization gives the splitter reliable structure

	without mutating the Marker output the project is still assessing, preserving reproducibility and the "do not silently rewrite source" stance. Scoping to observed patterns keeps it a best-effort heuristic rather than a general Markdown normalizer.
Impact	Adds the <code>preprocessing()</code> stage to the chunking pipeline in <code>Architecture.md</code> ; introduced and closed BUG-013 through BUG-015 during ground-truth validation; documented in Research topic 9. It refines rather than overrides ADR-006 and the "chunking is not a repair mechanism" principle: repair is limited to in-memory structural normalization that feeds boundary detection, not authoritative correction of Marker's output, and remains bounded by defects such as BUG-012.

ADR-008 · Run Marker in balanced mode, sequentially — page batching and parallelism rejected

Field	Detail
Decision	Pin <code>codes/marker_test.py</code> to Marker's <code>balanced</code> mode and convert each PDF whole, in one <code>PdfConverter</code> call, one file at a time. Do not batch pages, and do not run converters concurrently. The parallel/batching implementation was written, measured, and reverted — this ADR exists so it is not attempted again.
Date	2026-07-21
Context	The driving goal was better chunk quality by removing running headers/footers from Marker's Markdown, not throughput. <code>balanced</code> mode on a 250-page RFT was slow enough to be abandoned mid-run, which prompted a switch to <code>fast</code> and then to page batching with concurrent workers as a way to make <code>balanced</code> affordable.
Options Considered	<code>fast</code> mode (rejected on quality : roughly as quick as the old pipeline and headers were gone, but it merged separate list points together, degrading chunking — a structural defect chunking cannot recover, cf. BUG-012) · Per-worker <code>create_model_dict()</code> as originally proposed (rejected on design : <code>SuryaInferenceManager</code> is documented "construct once per process" and every predictor is a thin client of a shared server, so a second dict races to spawn a second llama-server on the same GPU) · Shared model dict, two concurrent batch workers (rejected on stability : aborts the llama-server, BUG-016) ·

	Shared model dict, batches serial (rejected on speed : 32% slower than not batching) · Whole-document balanced , sequential (chosen)
Chosen Solution	Whole-document balanced conversion. Measured on pages 0–59 of the RFT corpus, RTX 5050 Laptop (8 GB): whole document 90.1s; three 20-page batches serial 118.7s; three 20-page batches across two workers aborts the llama-server. All batching code was removed from marker_test.py ; the script is back to one converter and a simple loop.
Rationale	Document-level threading duplicates parallelism that already exists one layer down: surya runs a single llama-server with SURYA_INFERENCE_PARALLEL (default 8) slots and coalesces pages across all in-flight requests. Splitting into 20-page calls shrinks the pool of pages available to coalesce and re-pays provider/converter setup per batch, so it is slower <i>even before</i> the crash — the batching code was pure overhead. Note SURYA_INFERENCE_PARALLEL=8 is a no-op override: it equals the backend's existing DEFAULT_PARALLEL .
Impact	marker_results/ output is not comparable to prior fast -mode runs (see the pipeline-mixing caveat in CLAUDE.md — regenerate the tree before using it as a baseline). Opens BUG-016. Leaves the original header/footer goal unaddressed by this ADR : the actual lever is MarginaliaProcessor config on the converter, which is independent of mode and of scheduling — see Research and the ClassName_attr config note in CLAUDE.md. Accept that a 500-page balanced run is simply slow on this hardware; there is no scheduling trick that avoids it.

ADR-009 · Upgrade Marker to 2.0.0 and run its VLM through a local llama.cpp server

Field	Detail
Decision	Move <code>.venv-marker</code> from <code>marker-pdf==1.10.2</code> to <code>2.0.0</code> (surya-ocr 0.22.1) to obtain <code>MarginaliaProcessor</code> , and serve the VLM through a locally installed <code>llama-server</code> binary rather than the auto-selected vllm/Docker backend. Accept the resulting external runtime dependency and the three environment fixes it forced.
Date	2026-07-21
Context	The goal was chunk quality: running headers and footers were entering the reading stream in <code>marker_results/</code> and degrading chunk boundaries. Under 1.10.2 nothing could remove them, because <code>keep_pageheader_in_output=False</code> only suppresses blocks the layout model actually classified as <code>PageHeader</code> ; anything labelled <code>Text</code> or <code>SectionHeader</code> survives regardless (Research topic 6). Marker 2.0.0 adds <code>MarginaliaProcessor</code> to the default PDF pipeline, which relabels running headers/footers positionally — by margin zone, page-extremality, block height, and text length — and is therefore not dependent on the layout model's classification. A local geometric processor was designed against 1.10.2 as an alternative and discarded once the upstream version was confirmed installed; the shipped implementation is better guarded.
Options Considered	Stay on 1.10.2 and register a custom geometric processor (rejected: duplicates upstream work with weaker guards, and Marker's <code>processor_list</code> path needs importable dotted paths, forcing a converter subclass) · Relabel low-confidence blocks with <code>BlockRelabelProcessor</code> (rejected: document-wide, positionally unaware, and its confidence comparison is inverted from the intuitive reading, so it demotes genuine mid-page section headers) · Tune <code>IgnoreTextProcessor</code> 's repetition thresholds (rejected as insufficient alone: it inspects only the first and last structured block per page, so a multi-block header is only ever partially caught) · Upgrade to 2.0.0 with the default vllm/Docker backend (rejected: a ~10 GB image, WSL2 GPU passthrough, and vllm defaults of 0.85 GPU-memory utilization against an 18,000-token model length on 8 GB of VRAM) · Upgrade to 2.0.0 with the native llama.cpp backend (chosen)
Chosen Solution	<code>marker-pdf==2.0.0</code> ; <code>SURYA_INFERENCE_BACKEND=llamacpp</code> ; <code>llama.cpp</code>

	b10068 (CUDA 13.3 build plus cudart) installed to %LOCALAPPDATA%\llama.cpp and added to the user PATH, with a LLAMA_CPP_BINARY fallback in codes/marker_test.py for shells that predate the PATH edit; SURYA_GUIDED_LAYOUT=false; Torch restored to 2.13.0+cu130 and the orphaned torchaudio removed. MarginaliaProcessor is left at its shipped defaults (header_zone=0.08, footer_zone=0.13, max_height_frac=0.035, max_chars=150) — no tuning proved necessary.
Rationale	The upgrade is the only route to positional marginalia removal, which was the actual objective. The backend choice follows from hardware: surya auto-selects vllm on any NVIDIA GPU (BUG-017), and vllm targets server-class cards. llama.cpp runs the quantized GGUF natively with no container. cu130 is the only PyTorch index carrying torch 2.13.0 for this Blackwell (sm_120) GPU — cu128 stops at 2.9.1 — so the CUDA restore was a pure local-tag change with no dependency drift. Verified outcome: in the regenerated corpus the running-header line that previously opened the document is gone, and the only surviving RFT Number: occurrences are legitimate form fields inside tables.
Impact	Converts the Marker leg of this harness from "pip install and run" into "operate a local inference server": conversion now depends on an external binary, a spawned server process, model downloads (~1.5 GB of GGUF plus a 136 MB rf-detr checkpoint), and server lifecycle management including orphaned processes holding VRAM after an interrupted run. Forced BUG-017 through BUG-019 and surfaced BUG-020. Introduces the device-derived mode default that ADR-008 then pins. Invalidates every pre-upgrade file in marker_results/: the tree may hold output from 1.10.2, 2.0.0-fast, and 2.0.0-balanced, which are three different extraction pipelines and are not comparable (Research topic 10). Setup steps are recorded in CLAUDE.md; no tracked dependency manifest captures them.

ADR-010 · Chunk by packing a structural tree, not by selecting boundaries against a size limit

Field	Detail
Decision	Rewrite <code>temp_split()</code> as a two-phase algorithm: parse the document once into a tree of structural units (sections nested by ATX heading level; within each section its own tables, lists and paragraphs), then pack those units greedily into chunks. Retire the rule-based boundary selector <code>structural_cut()</code> and its supporting helpers entirely.
Date	2026-07-22
Context	The hand-authored rule set from 2026-07-17 had been extended six times against real Marker output and was still producing two classes of defect the user reported directly: chunks far below <code>CHUNK_SIZE</code> sitting next to content that would have fitted (a 64-character opening chunk), and sections torn in half at a subsection boundary (clause 5.4 split between <code>Technical Bids</code> and <code>Commercial Bids</code>). See BUG-022.
Options Considered	Add a minimum-fill rule to <code>structural_cut()</code> (rejected: it would be the seventh patch to a rule set whose rules already contradict one another — <code>list_section_after_heading()</code> deliberately emits an under-full chunk, so a global minimum-fill rule would need carve-outs for its own siblings) · Keep boundary selection but repair the deepest-ATX-level heuristic (rejected: the heuristic is a proxy for section ownership that only works when heading levels are well-formed, which Marker's are not — BUG-021 — so it would stay one malformed document away from the same failure) · Parse-then-pack over a structural tree (chosen)
Chosen Solution	<code>build_tree()</code> produces <code>_Node</code> units; <code>pack()</code> fills a chunk with whole sibling units until the next would not fit, then moves that unit forward intact; <code>split_node()</code> opens a unit up only when it cannot fit an empty chunk by itself, and only one level at a time. A heading always travels with the first unit beneath it (<code>extend_start()</code>), so no depth of splitting can strand one. A pending buffer holding only headings, or one sitting in front of a unit about to be opened anyway, is handed into that unit rather than published early (<code>lead_in_fits()</code>). Oversized lists split on top-level pointers, keeping nested children with their parent pointer.
Rationale	The user's stated intent — "fill each chunk with as many

	characters as <code>CHUNK_SIZE</code> permits, but never separate a heading from its content or split a logical section; if the complete next section cannot fit, move the entire section to the next chunk" — is literally the definition of greedy packing over a containment tree. Expressing it that way makes both reported defects unrepresentable rather than patched: a chunk closes only when the next whole unit does not fit, and a unit is only ever divided along boundaries the document itself declares. It also removed 202 lines and eleven helpers whose interactions were the actual source of the bugs.
Impact	Chunk boundaries change document-wide; earlier <code>chunk-runs/</code> reports are not comparable. Chunk count on the two-document corpus fell from 1,949 to 1,051 at <code>CHUNK_SIZE=1600</code> as chunks became fuller. Verified lossless by whitespace-normalised round trip across <code>marker_results/</code>, <code>docling_results/</code> and <code>unstructured_results/</code> (nine files). The rewrite also dropped the predecessor's trailing-heading guard, which had to be restored as BUG-023. The splitter remains experimental — there is still no test suite, and verification means re-running <code>ingest.py</code> and reading the report. Supersedes the algorithm documented in Research topic 8; ADR-011 then removes the table-size exemption this ADR inherited.

ADR-011 · Bound tables by `CHUNK_SIZE` too, re-serializing them with repeated headers

Field	Detail
Decision	Remove the standing exemption that let a Markdown table be emitted whole at any size. A table that does not fit is split into several chunks, each re-serialized with the headers that apply to the cells it carries. Splitting is by row when the table has column headers, both, or neither; by column when it has only row headers; and descends to cell groups and then to content inside a single cell when a row, column, or cell is itself too large.
Date	2026-07-22
Context	Every prior chunker in this project treated a table as one indivisible retrieval unit, on the grounds that a header row separated from its data is worse than an oversized chunk — the conflict Research topic 7 had already identified as unresolvable in the abstract. Measured against the real corpus that exemption cost 38 chunks above the limit, the largest 10,071 characters, well past the context budget any downstream retrieval step would allocate to a single chunk. The guarantee was protecting a unit that could not actually be used. The user supplied nine explicit rules covering how a table should be divided.
Options Considered	Keep the exemption (rejected: an unusable chunk is not a preserved one) · Split tables by row without repeating headers (rejected: every piece after the first loses the column names, which is exactly the failure the exemption existed to prevent) · Slice table pieces out of the source verbatim (rejected: a verbatim slice cannot carry a repeated header, and Marker pads columns to the widest cell — alignment rows of up to 897 characters occur here — so most of a slice's budget is padding) · Re-serialize with normalized padding and repeated headers (chosen)
Chosen Solution	<code>split_table()</code> parses the table into a header stack, an alignment row and data rows, then dispatches on which headers exist. Padding and the alignment row are normalized on output; the size limit is measured on the emitted chunk, not on the source it came from. Orientation is decided by <code>has_column_headers()</code> and <code>has_row_headers()</code> , the latter a documented heuristic since Markdown has no row-header syntax. Fallbacks descend one level at a time: whole rows → cell groups within a row (<code>split_across_columns</code>) or rows within a column (<code>split_across_rows</code>) → content within one cell (<code>split_cell</code> ,

	preferring paragraph, then list-item, then sentence, then word boundaries). Any heading introducing the table repeats on every piece, since each piece is meant to stand alone. No fragment index, table identity, or generated positional label is added — only real headers are repeated.
Rationale	Repeating the headers is what makes splitting safe: the reason tables were exempt is that a bare row block is uninterpretable, and a repeated header removes that objection for a few dozen characters per chunk. Normalizing padding is not cosmetic — the RFT summary table is 5,922 source characters carrying 1,823 characters of content — so normalization alone brings many tables back under the limit and satisfies rule 1 with no split at all. <code>has_row_headers()</code> only chooses between splitting axes and whether a row label repeats, so a wrong guess degrades a chunk rather than corrupting one.
Impact	No chunk in the corpus now exceeds <code>CHUNK_SIZE</code>, down from 38. Chunk count rose from 1,051 to 1,069 at <code>CHUNK_SIZE=1600</code>. Split tables are no longer byte-identical to their source — padding is normalized, alignment rows are rewritten to <code>---</code>, and headers appear more than once — so the strict round-trip check used since 2026-07-20 no longer applies to them and was replaced by a per-line survival check: every source line, normalized to its rendered form, must appear in some chunk. Content can now be lost only inside a single cell that exceeds the limit on its own, which no table in the current corpus does. Supporting changes: <code>table_cells()</code> now honours escaped and code-span pipes so column indices are correct, and <code>table_spans()</code> extends backwards over rows above the alignment row so multi-level header stacks are captured and repeated in full. Removes the exemption ADR-010 inherited; closes the oversized-table policy gap left open by Research topic 7.

ADR-012 · Revert Marker to 1.10.2 and abandon the 2.0.0 VLM-server pipeline

Field	Detail
Decision	Return <code>.venv-marker</code> to <code>marker-pdf==1.10.2</code> and restore <code>codes/marker_test.py</code> to its pre-upgrade form (<code>c9f712a</code>) — a bare <code>PdfConverter(artifact_dict=create_model_dict())</code> . This reverses ADR-009, moots ADR-008's <code>balanced</code> pin, and discards the same-day recursion and worker-pool instrumentation along with it.
Date	2026-07-23
Context	2.0.0 was adopted for one reason: <code>MarginaliaProcessor</code> , which removes running headers and footers positionally and which 1.10.2 cannot do at any configuration (ADR-009; Research topic 6, topic 10). That objective was met. What followed was two days of failures the pipeline change had introduced — Docker/vllm backend selection, a GBNF grammar rejection, an ABI-orphaned <code>torchaudio</code> , a renderer recursion crash, a dead <code>pdftext</code> worker (BUG-017 through BUG-020, BUG-024, BUG-025) — and then output that was worse than what the upgrade replaced. The deciding evidence is <code>marker_results/src2.md</code> : a 154-page source rendered as 244 lines, 50 MB, with 83 of them a single 215-column table and 9 headings in the whole document (BUG-026). The user's stated ground for reverting was output quality, and the tree corroborates it.
Options Considered	Stay on 2.0.0 and quarantine the bad document (rejected: the harness exists to compare loaders, and a pipeline that can silently turn a 154-page contract into one table is not a baseline — the failure is not detectable from the batch summary, which counted it as converted) · Stay on 2.0.0 and keep the recursion/worker instrumentation as permanent guards (rejected: both are workarounds for defects the older pipeline does not have, and each adds machinery — a spawned thread with a 128 MB stack, a page-batching fallback — to a script whose value is being small enough to read) · Pin 2.0.0 to <code>fast</code> mode (rejected earlier on structural grounds in ADR-008: it merges separate list points, which chunking cannot recover) · Revert to 1.10.2 and accept losing <code>MarginaliaProcessor</code> (chosen)
Chosen Solution	<code>uv pip install --python .venv-marker\Scripts\python.exe "marker-pdf==1.10.2"</code> , which rolled back the transitive stack with it (surya-ocr 0.22.1 → 0.17.1, pdftext 0.7.1 → 0.6.3, pypdfium2

	5.10.1 → 4.30.0, transformers 5.14.1 → 4.57.6, plus the anthropic/openai/google-genai/huggingface-hub clients). <code>git checkout c9f712a -- codes/marker_test.py</code>, committed as aa1627f. Torch was deliberately left untouched at 2.13.0+cu130 and re-verified with an actual matmul; <code>tiktoken</code> survived, so <code>chunking/</code> is unaffected.
Rationale	The upgrade traded one known, bounded defect (running headers entering the reading stream, which <code>preprocessing()</code> and the packer already cope with) for an unbounded one (arbitrary structural collapse with no signal at the batch level). Between a defect you can see in every document and a defect that appears in one document out of three and is invisible until you open the file, the first is the better position for a comparison harness. <code>codes/marker_test.py</code> reverting past the 2.0.0 adaptation layer is not incidental damage — <code>SURYA_INFERENCE_BACKEND</code>, the <code>LLAMA_CPP_BINARY</code> fallback and <code>SURYA_GUIDED_LAYOUT=false</code> exist only to make surya 0.22.1 run, and are inert or harmful once 0.17.1 is installed.
Impact	Header/footer removal is lost. <code>MarginaliaProcessor</code> does not exist in the installed 1.10.2 tree (verified), so ADR-009's delivered outcome is reversed and Research topic 6's conclusion — that 1.10.2 cannot reach a running header the layout model labelled <code>Text</code> — governs again the moment the corpus is regenerated. The Marker leg also stops being an operated service: no <code>llama-server</code> binary, no spawned inference servers, no GGUF downloads, no orphaned VRAM. Two defects survive the revert while their fixes do not. <code>markdownify</code> over <code>html.parser</code> is the renderer path in 1.10.2 as well (BUG-024), and <code>pdftext 0.6.3</code> has the identical <code>min(workers, pages // 10)</code> pool sizing with the same <code>pdftext_workers = 4</code> provider default (BUG-025) — Marker's own CLI and server front-ends force it to 1, and <code>codes/marker_test.py</code> now does not. <code>marker_results/</code> no longer matches the installed version at all: all five files were produced by 2.0.0 (two <code>balanced</code> on 2026-07-22, three on 2026-07-23), so the whole tree must be regenerated before it is a baseline for anything, and regenerating it will reintroduce running headers into the chunking corpus. <code>docs/</code>, <code>CLAUDE.md</code> and <code>README.md</code> all documented the 2.0.0 setup as current until this entry. Supersedes ADR-009; ADR-008's mode pin is moot, since 1.10.2 has no <code>mode</code> concept, though its rejection of page batching and concurrent converters stands on independent grounds.